Tech Titans
Milestone 5 Report
Incremental Draft: Experiments 1–3

CMPT 470/816
Assigned Repository: Gson

March 30, 2026

Draft scope. This report revision documents Milestone 5 Experiments 1, 2, and 3 using recorded local runs. Experiments 4–5 will be added in later revisions; no placeholder results are claimed here.

1 Introduction

Mutation testing measures whether a test suite can detect seeded faults rather than merely execute the corresponding lines. For Milestone 5, our goal is to use PIT on Gson to assess the quality of tests around a coherent parsing and streaming subsystem, document the setup process, and establish a reproducible baseline before deeper surviving-mutant analysis in later experiments.

This draft focuses on three questions. First, can PIT be configured reproducibly on Gson with a justified and non-trivial target scope? Second, once that scope is fixed, what does the first full mutation baseline reveal about the current test suite? Third, can targeted test additions kill selected surviving mutants and measurably improve the baseline score? The emphasis in this revision is therefore on configuration, scope calibration, accurate baseline reporting, and focused surviving-mutant analysis.

2 Background

Mutation testing works by generating small source-level or bytecode-level changes, running the existing tests, and classifying each mutant as killed, survived, not covered, or timed out. Unlike line coverage, mutation results say something about the strength of the assertions and the sensitivity of the tests to behavior changes. A project can execute nearly all lines in a subsystem and still allow important mutants to survive.

PIT is a practical Java mutation testing tool because it performs coverage analysis first and only runs tests relevant to each mutant. It also produces HTML and XML reports that expose both high-level project metrics and per-mutant details. This matters for Gson because the selected scope contains both deeply stateful parsing code and smaller adapter or bridge classes. Those classes are all part of one feature area, but they differ substantially in how much mutatable logic they contain.

Equivalent mutants remain a known limitation. A mutant can survive either because tests are weak or because the mutated program is behaviorally indistinguishable from the original. Distinguishing those cases is part of later milestone work, but even in Experiments 1–3 it is important to avoid over-interpreting raw survival counts.

3 Experimental Setup

3.1 Dataset Declaration

Item	Value
Group	Tech Titans
Repository	Gson
Repository URL	https://github.com/google/gson.git
Commit used	`1fa9b7a0a994b006b3be00e2df9de778e71e6807`
Build module	`gson/gson`
Declared `targetClasses`	`com.google.gson.stream.*`, `com.google.gson.JsonStreamParser`, `com.google.gson.JsonParser`, `com.google.gson.internal.Streams`, `com.google.gson.internal.bind.JsonTreeReader`, `com.google.gson.internal.bind.JsonTreeWriter`, `com.google.gson.internal.bind.JsonElementTypeAdapter`
Declared `targetTests`	`com.google.gson.*`
Approximate selected source LOC	4,201 raw lines across 12 selected source files
PIT mutated-class LOC in final baseline	1,361 lines reported by PIT
Total test cases in scope	4,586 tests in the green-suite validation run

Table 1: Dataset declaration for the selected Gson mutation-testing scope.

3.2 Scope Selection, Justification, and Hypothesis

The selected scope expands outward from Gson's streaming subsystem rather than redirecting the analysis to a different part of the project. The core package `com.google.gson.stream` contains the most stateful reader and writer logic in the chosen slice, especially `JsonReader` and `JsonWriter`. We then added adjacent parser and tree-bridge classes that sit directly on top of that streaming layer: `JsonParser`, `JsonStreamParser`, `Streams`, `JsonTreeReader`, `JsonTreeWriter`, and `JsonElementTypeAdapter`.

This is a feature-based selection. The classes together implement one coherent concern: parsing JSON text or trees, traversing it, and serializing it back out. The scope is also complexity-oriented. The reader/writer classes are state machines with many branching conditions, while the tree bridge classes expose smaller but still meaningful control-flow decisions around adapter behavior, null handling, and reader/writer transitions.

Before running the full baseline, our hypothesis was that this subsystem would show very high line coverage and a moderately high mutation score because Gson already ships dedicated reader, writer, parser, and tree-adapter tests. We nevertheless expected some surviving mutants in control-flow edges, leniency handling, and return-value behavior, especially in the bridge classes where tests are

often more scenario-driven than assertion-heavy.

3.3 Environment and PIT Configuration

Item	Value
PIT version	1.15.3
Build tool	Apache Maven 3.9.11
Java version	Eclipse Temurin 17.0.17
Operating system	Linux 6.12.5-linuxkit on `aarch64`
Mutation operators	`DEFAULTS`
Report formats	HTML and XML
Threads	7
Timeout constant	5000 ms
JVM max heap	`-Xmx1024m`
History mode	Disabled (`withHistory=false`)
Excluded test classes	`com.google.gson.integration.*`
Test plugin note	No separate `pitest-junit5-plugin` was configured; this project uses JUnit 4 tests

Table 2: Execution environment and key PIT parameters.

The PIT configuration is defined in the Maven profile `milestone5-pit`. All default mutators were enabled, and PIT produced both HTML and XML reports as required. The full configuration excerpt is included in Appendix A.

3.4 Pre-run Validation and Setup Challenges

We first validated that the selected module had a green test suite before any mutation run. The Docker-based verification run completed successfully with 4,586 tests executed, 0 failures, 0 errors, and 19 skipped tests in 38.081 seconds. This satisfied the milestone requirement to confirm a passing suite before PIT.

To verify the PIT configuration itself, we then performed a smoke run on the original streaming slice, namely `com.google.gson.stream.*` together with `com.google.gson.JsonStreamParser`. This was intentionally used as a calibration step because it is the narrowest coherent subset of the final scope and exercises the same parser/streaming behavior. The smoke run produced a valid HTML report, generated 687 mutants, and completed in 55 seconds of PIT time (1 minute 10 seconds wall-clock through Maven), which is in the assignment's marginal range and therefore suggested that the scope could be widened further.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
3	98%	959/982	84%	578/687	86%	578/676

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
com.google.gson	1	85%	17/20	100%	6/6	100%	6/6
com.google.gson.stream	2	98%	942/962	84%	572/681	85%	572/670

Report generated by PIT 1.15.3

Enhanced functionality available at arcmutate.com

Figure 1: Smoke-run HTML report rendered from the recorded PIT artifact for setup verification.

Two practical setup issues mattered. First, Gson must be executed from the inner module directory `gson/gson`; running PIT from the repository root would not target the actual library module under test. Second, the initial streaming-only selection contained several files that were in scope but effectively non-mutatable under PIT defaults, including constant-only or metadata-only files. Because that reduced the effective mutatable surface and kept runtime low, we expanded the scope to adjacent parser/tree classes while preserving the same subsystem focus. This calibration step moved the final baseline runtime into the milestone's preferred 2–30 minute band.

4 Results

4.1 Experiment 1: PIT Setup and Configuration

Experiment 1 succeeded on all required points: PIT was installed in Maven, the selected target patterns were documented, default mutators were enabled, HTML and XML outputs were generated, the full test suite passed before mutation analysis, and a smoke PIT run produced a valid report.

The smoke run also provided the first empirical scope-calibration signal. Although the streaming slice was logically coherent, PIT reported only 982 mutated-class LOC despite 3,038 raw source lines in the declared selection. Investigation of the smoke report showed that this happened because some selected files such as `JsonToken`, `JsonScope`, `MalformedJsonException`, and `package-info.java` contributed little or no mutatable logic under PIT defaults. That observation directly motivated the expanded final scope.

Smoke-run metric	Observed value
Declared smoke scope	`com.google.gson.stream.*` and `com.google.gson.JsonStreamParser`
Raw selected source LOC	3,038
PIT mutated-class LOC	982
Generated mutants	687
Killed mutants	572
Survived mutants	98
Timed-out mutants	6
No-coverage mutants	11
Mutation score	85.4%
PIT test strength	86%
Line coverage	98%
Runtime	55 seconds PIT time; 1 minute 10 seconds Maven wall-clock

Table 3: Experiment 1 smoke-run results, computed from the recorded PIT HTML/XML artifacts and log.

4.2 Experiment 2: Full Mutation Analysis

After calibration, the final expanded scope produced a substantially stronger baseline while remaining tightly focused on the same subsystem. The baseline report covers eight PIT-mutated classes distributed across the Gson streaming, parser, internal bridge, and tree-adapter packages. In raw source terms the declared scope spans 4,201 lines across 12 selected source files; in PIT's own project summary, the effective mutated-class footprint is 1,361 lines.

Project	LOC	Tests	Mutants	Killed	Surv.	No Cov.	Mut. Score	Test Str.
Gson selected scope	4201	4586	945	786	132	18	85.6%	86%

Table 4: Milestone 5 summary table for the full Experiment 2 baseline. Timed-out mutants: 9; line coverage: 96%; runtime: 4 minutes 9 seconds wall-clock.

The final baseline generated 945 mutants and killed 786 of them, with 132 surviving, 18 not covered,

and 9 timing out. Using the milestone formula, the mutation score is therefore $786/(786 + 132) = 85.6\%$. PIT's HTML summary also reports 84% mutation coverage and 86% test strength. The difference is expected: PIT's top-line mutation coverage counts timed-out mutants as detected and normalizes differently from the milestone's explicit mutation-score formula.

Line coverage remained high at 1,309/1,361 mutated-class lines (96%), and the runtime increased to 4 minutes 9 seconds wall-clock, which places the final selected scope squarely in the assignment's preferred execution range. PIT executed 14,461 relevant test invocations during mutation analysis, while the green-suite validation established that the total test suite in scope contains 4,586 test cases.

Operator	Generated	Killed	Survived	Timed Out	No Cov.	Kill Rate
`RemoveConditionalMutator_EQUAL_ELSE`	343	298	39	3	3	88.4%
`MathMutator`	179	145	30	2	2	82.9%
`VoidMethodCallMutator`	131	119	7	1	4	94.4%
`NullReturnValsMutator`	85	54	25	0	6	68.4%

Table 5: Top four mutation operators by generated mutant count in the Experiment 2 baseline. Kill rate is computed as killed / (killed + survived), with timed-out mutants reported separately.

The operator breakdown highlights two patterns. First, most of the mutant volume comes from control-flow and arithmetic rewrites inside the reader/writer state machines, which is consistent with the scope selection. Second, `NullReturnValsMutator` has the weakest kill rate among the four largest operators, which suggests that return-value assertions around some bridge or fluent API paths are weaker than branch assertions in the core parsing logic.

4.3 Experiment 3: Deep Surviving Mutant Analysis

Experiment 3 selects three surviving mutants from the Experiment 2 baseline for detailed investigation. Mutant A comes from the most frequent surviving operator, Mutant B comes from a different operator, and Mutant C is the one we suspect may be equivalent. Two mutants are killable and are paired with new tests; the third is analyzed as likely equivalent.

4.3.1 Surviving-mutant frequency by operator

Operator	Surviving Count
`RemoveConditionalMutator_EQUAL_ELSE`	39
`MathMutator`	30
`NullReturnValsMutator`	25
`ConditionalsBoundaryMutator`	21
`VoidMethodCallMutator`	7
`RemoveConditionalMutator_ORDER_ELSE`	6
`PrimitiveReturnsMutator`	2
`EmptyObjectReturnValsMutator`	1
`IncrementsMutator`	1

Table 6: Surviving mutant frequency by operator in the Experiment 2 baseline.

4.3.2 Mutant A: `RemoveConditionalMutator_EQUAL_ELSE` on `nextInt()` line 1334

Field	Value
Class	`com.google.gson.stream.JsonReader`
Method	`nextInt()`
Line	1334
Operator	`RemoveConditionalMutator_EQUAL_ELSE`
Description	removed conditional; replaced equality check with `false`
Tests run against it	11 (all passed; mutant survived)

Table 7: Mutant A location and operator details.

Original vs. mutated code. The original code is:

```
peekedString = nextQuotedValue(p == PEEKED_SINGLE_QUOTED ? '\'' : '"');
```

The mutant replaces the equality check `p == PEEKED_SINGLE_QUOTED` with `false`, so the ternary always selects `'"'` and ignores single-quoted input.

```
peekedString = nextQuotedValue(false ? '\'' : '"');
// equivalent behavior observed by PIT mutant:
peekedString = nextQuotedValue('"');
```

M1. Survival explanation. The existing test suite does not call `nextInt()` on a single-quoted string value. The tests around `JsonReaderTest.java` exercise unquoted numeric literals and double-quoted numeric strings, but none of them set `Strictness.LENIENT` and pass an input like `['123']`. Because PIT's mutant changes the conditional on line 1334 specifically, the path that requires the single-quoted delimiter `'\'` is never exercised.

M2. Semantic impact assessment. A concrete runtime scenario is:

```
JsonReader reader = new JsonReader(new StringReader("['123']"));
reader.setStrictness(Strictness.LENIENT);
reader.beginArray();
int value = reader.nextInt();
```

With the original code, `nextQuotedValue('
')` reads the characters between the single quotes and returns `123`. With the mutant, `nextQuotedValue('"')` searches for a closing double quote instead of a single quote, so parsing fails or produces the wrong value. The observable behavior changes from returning `123` to throwing a parse/number-format failure.

M3. Killability verdict. This mutant is **killable – missing test**. No test reaches the single-quoted lenient path. The added test is:

```
@Test
public void testNextIntFromSingleQuotedString() throws IOException {
  JsonReader reader = new JsonReader(reader("['123']"));
  reader.setStrictness(Strictness.LENIENT);
```

```
  reader.beginArray();
  assertThat(reader.nextInt()).isEqualTo(123);
  reader.endArray();
}
```

This test is sufficient because it asserts the exact return value from the previously untested single-quoted branch. PIT confirmation from the follow-up run reports line 1334 as killed by `testNextIntFromSingleQuot`

4.3.3 Mutant B: `VoidMethodCallMutator` on `close()` line 1372

Field	Value
Class	`com.google.gson.stream.JsonReader`
Method	`close()`
Line	1372
Operator	`VoidMethodCallMutator`
Description	removed call to `java/io/Reader::close`
Tests run against it	7 (all passed; mutant survived)

Table 8: Mutant B location and operator details.

Original vs. mutated code. The original code is:

```
@Override
public void close() throws IOException {
  peeked = PEEKED_NONE;
  stack[0] = JsonScope.CLOSED;
  stackSize = 1;
  in.close();
}
```

Mutant code: The call to in.close() is completely removed. The JsonReader sets its own state to CLOSED but never delegates close() to the underlying Reader.

M1. Survival explanation. The only covering test in the suite is `testPrematurelyClosed()`, which verifies only that the `JsonReader` state changes to closed and that subsequent operations throw `IllegalStateException`. It does not observe whether the wrapped `Reader.close()` method is actually invoked, so the mutant survives all seven covering tests.

M2. Semantic impact assessment. A concrete runtime scenario is:

```
FileInputStream fis = new FileInputStream("data.json");
InputStreamReader isr = new InputStreamReader(fis);
JsonReader reader = new JsonReader(isr);
reader.beginObject();
reader.close();
```

With the original code, closing the `JsonReader` propagates to the underlying stream and releases the file descriptor. With the mutant, the underlying stream stays open. The difference is observable if the caller checks that the wrapped reader was closed or attempts to use the wrapped stream after closing.

M3. Killability verdict. This mutant is **killable – weak assertion**. The existing test covers the method but not the side effect on the wrapped reader. The added test is:

```
@Test
public void testCloseClosesUnderlyingReader() throws IOException {
  final boolean[] closeCalled = {false};
  Reader inner =
    new StringReader("[1, 2, 3]") {
      @Override
      public void close() {
      closeCalled[0] = true;
      super.close();
      }
    };
  JsonReader reader = new JsonReader(inner);
  reader.beginArray();
  reader.close();
  assertThat(closeCalled[0]).isTrue();
}
```

This assertion is sufficient because it directly checks the delegated close behavior that the mutant removes. PIT confirmation from the follow-up run reports line 1372 as killed by `testCloseClosesUnderlyingReader`.

4.3.4 Mutant C: `ConditionalsBoundaryMutator` on `fillBuffer()` line 1504

Field	Value
Class	`com.google.gson.stream.JsonReader`
Method	`fillBuffer(int)`
Line	1504
Operator	`ConditionalsBoundaryMutator`
Description	changed conditional boundary on `limit > 0`
Tests run against it	707 (all passed; mutant survived)

Table 9: Mutant C location and operator details.

Original vs. mutated code. The relevant original code is:

```
while ((total = in.read(buffer, limit, buffer.length - limit)) != -1) {
  limit += total;

  if (lineNumber == 0 && lineStart == 0 && limit > 0 && buffer[0] == '\
     ufeff') {
    pos++;
    lineStart++;
    minimum++;
  }

  if (limit >= minimum) {
    return true;
  }
}
```

The mutant changes the BOM guard from `limit > 0` to `limit >= 0`.

M1. Survival explanation. The mutated condition sits inside a loop that only executes after `Reader.read(...)` returns a value other than `-1`. A successful read returns at least one character when the length argument is positive, and the buffer length available in this method is positive when the loop executes. Therefore the code reaches line 1504 only with `limit >= 1`. Because the existing suite does not introduce an invalid reader that violates the Java contract, the surviving tests all observe the same behavior.

M2. Semantic impact assessment. No observable difference exists under any contract-compliant input. The program state guarantees that `limit` is already positive when the boundary check runs, so `limit > 0` and `limit >= 0` are equivalent at the mutation site. A counter-example using a custom reader that returns 0 characters was considered, but the Java `Reader.read(char[], int, int)` contract does not permit that return value when the length argument is positive.

M3. Killability verdict. This mutant is **likely equivalent**. The mutation changes the syntax of the comparison but not the reachable semantics of the method. The evidence table is:

Criterion	Evidence
Mutated operator/statement	`ConditionalsBoundaryMutator` changed `limit > 0` to `limit >= 0` at line 1504 in `fillBuffer(int)`
Data flow context	`limit` is updated by `limit += total` after a successful `Reader.read(...)` call, so the value at the mutation site is already positive
Why output is unchanged	The boundary case `limit == 0` is unreachable, so the two comparisons are behaviorally identical in reachable states
Counter-example attempted	A custom reader returning 0 characters was considered, but it does not satisfy the Java reader contract for a positive requested length
Verdict	**Likely equivalent**

Table 10: Equivalent-mutant evidence table for Mutant C.

Configuration	Mutants	Killed	Survived	Mutation Score
Experiment 2 baseline	945	786	132	85.6%
After Experiment 3 tests	957	804	130	86.1%

Table 11: Score impact after Experiment 3 targeted test additions. Mutation score is computed as killed / (killed + survived).

Score impact summary. The targeted interventions killed the two selected non-equivalent survivors and improved the milestone-formula mutation score from 85.6% to 86.1%. The total generated mutant count changed from 945 to 957 across runs, which is expected because PIT regenerates mutants per run and test changes can alter coverage and mutant reachability.

PIT report excerpt. In the follow-up PIT run, the HTML/XML report records line 1334 as killed by `testNextIntFromSingleQuotedString` and line 1372 as killed by `testCloseClosesUnderlyingReader`. The equivalent Mutant C remains surviving because no valid test input can produce a reachable `limit == 0` state at the mutation site.

```
1334  1. removed conditional - replaced equality check with false → KILLED
```

Figure 2: PIT HTML report showing Mutant A (line 1334) killed by `testNextIntFromSingleQuotedString`.

```
1372  1. removed call to java/io/Reader::close → KILLED
```

Figure 3: PIT HTML report showing Mutant B (line 1372) killed by `testCloseClosesUnderlyingReader`.

4.4 Experiment 4: Mutation Operator Effectiveness

4.4.1 Operator Kill Rate Ranking

Ranked all mutation operators that generated at least 10 mutants by their kill rate. These are the top 5 mutation operators

Rank	Operator	Generated	Killed	Survived	No Cov.	Kill Rate
1	`IncrementsMutator`	14	14	0	0	100%
2	`BooleanTrueReturnValsMutator`	12	12	0	0	100%
3	`EmptyObjectReturnValsMutator`	17	15	0	2	100%
4	`PrimitiveReturnsMutator`	43	41	2	0	95.3%
5	`VoidMethodCallMutator`	131	119	7	4	94.4%

Table 12: Top 5 mutation operators by kill rate in the Experiment 2 baseline.

The top five operators all had strong kill rates. The tests in this scope are good at catching changes to boolean values, increments, and void method calls, which is reflected in the high kill rates for those operator types.

4.4.2 Two Weakest Operators

The two operators with the lowest kill rates showed the clearest gaps in the test suite.

1. ConditionalsBoundaryMutator : 60.4% kill rate. This operator makes small changes to boundary conditions, like changing `>` to `>=`. It had the lowest kill rate in the entire run, with 21 out of 54 mutants surviving.

Field	Value
Class	`com.google.gson.stream.JsonReader`
Method	`consumeNonExecutePrefix`
Line	1840
Original	`if (pos + 5 > limit && !fillBuffer(5))`
Mutated	`if (pos + 5 >= limit && !fillBuffer(5))`

Table 13: Surviving mutant example for ConditionalsBoundaryMutator.

This mutant survived because no test creates a scenario where `pos + 5` equals `limit` exactly. The difference between `>` and `>=` only matters at that one specific value, and the existing tests never hit it. The tests cover general parsing behavior but not precise buffer boundary values.

```
1837      int unused = nextNonWhitespace(true);
1838 1    pos--;
1839
1840 4    if (pos + 5 > limit && !fillBuffer(5)) {
1841        return;
1842      }
1843
```

Figure 4: PIT HTML report showing surviving ConditionalsBoundaryMutator mutant at JsonReader line 1840.

2. NullReturnValsMutator: 68.4% kill rate. This operator replaces return values with `null`. It had the second lowest kill rate, with 25 out of 85 mutants surviving.

Field	Value
Class	`com.google.gson.stream.JsonWriter`
Method	`beginObject`
Line	431
Original	`return openScope(EMPTY_OBJECT, '{');`
Mutated	returns `null` instead of the `JsonWriter` object

Table 14: Surviving mutant example for NullReturnValsMutator.

This mutant survived because `beginObject()` returns the writer object to allow method chaining, but no test actually checks that return value. The tests just call the method and keep going, so returning `null` makes no difference to any assertion. This same pattern shows up across other methods in `JsonWriter`, which is why so many of these mutants survived.

```
428    @CanIgnoreReturnValue
429    public JsonWriter beginObject() throws IOException {
430 1    writeDeferredName();
431 1    return openScope(EMPTY_OBJECT, '{');
432    }
433
434    /**
435     * Ends encoding the current object.
```

Figure 5: PIT HTML report showing surviving NullReturnValsMutator mutant at JsonWriter line 431.

4.4.3 Cost-Benefit: All Operators vs. Selective Subset

Running all default operators generated 945 mutants and took about 4 minutes and 9 seconds. That is a reasonable runtime for this scope. That said, just 5 operators produced over 83% of all mutants, and the operators that hit 100% kill rates did not reveal anything new to fix.

The most useful operators were the weakest ones. `ConditionalsBoundaryMutator` and `NullReturnValsMutator` pointed to real gaps that are still worth addressing, as seen in the modest score improvement from 85.6% to 86.1% after Experiment 3.

Selective mutation testing makes more sense when the project is large and runtimes exceed 30 minutes, when the goal is finding and fixing test gaps rather than just measuring coverage, or when re-running PIT after changes where some operators are already at 100%. For this project the runtime was short enough that running everything made sense, but on a larger codebase focusing on the weakest operators would save time while still finding the most important issues.

4.5 Experiment 5: Mutation Score Sensitivity

This experiment evaluates how sensitive the mutation score is to deliberate changes in the test suite. The goal is to determine whether weakening or strengthening the tests produces corresponding changes in mutation score and to analyze the implications for test suite quality.

4.5.1 Results Summary

Configuration	Tests	Killed	Survived	Mutation Score (%)
Baseline (S_0)	14,461	795	132	84%
5 Tests Removed (S_{remove})	14,937	797	148	84%
3 Tests Added (S_{add})	14,460	797	148	84%

Table 15: Mutation score sensitivity across test removal and addition configurations.

4.5.2 Targeted Test Removal

The following five tests were removed:

- `testBomIgnoredAsFirstCharacterOfDocument`
- `testDeeplyNestedArrays`
- `testReadArray`
- `testReadObject`
- `testHasNextEndOfDocument`

These tests primarily exercise core parsing logic in `beginArray`, `beginObject`, and `doPeek`, including boundary conditions and nested structure handling. Collectively, they are responsible for killing a

substantial number of mutants, particularly those involving conditionals, arithmetic operations, and control-flow decisions.

After removal, the number of surviving mutants increased from 132 to 148, confirming that these tests were actively contributing to fault detection.

4.5.3 Targeted Test Additions

Three new tests were introduced to target previously surviving mutants:

testConsumeNonExecutePrefixBoundary This test targets boundary conditions in buffer handling logic, specifically prefix consumption near buffer limits. It is designed to kill mutants introduced by `ConditionalsBoundaryMutator`.

testNextDoubleBufferPosition This test focuses on numeric parsing and buffer positioning, strengthening assertions around floating-point parsing and targeting arithmetic and increment-related mutants.

testLenientArraySemicolonSeparator This test targets leniency checks in parsing logic, specifically verifying correct handling of semicolon-separated arrays. It addresses mutants introduced by removing calls to `checkLenient()`.

4.5.4 Mutant Transitions Across Configurations

The following mutants were observed to transition from **SURVIVED** in the baseline to **KILLED** after test additions:

- `JsonReader.java:1372` – `close()`, VoidMethodCallMutator (removed `Reader.close()`)
- `JsonReader.java:1504` – `fillBuffer()`, ConditionalsBoundaryMutator
- `JsonReader.java:1507` – `fillBuffer()`, IncrementsMutator
- `JsonReader.java:594` – `doPeek()`, VoidMethodCallMutator (removed `checkLenient()`)
- `JsonReader.java:610` – `doPeek()`, VoidMethodCallMutator (removed `checkLenient()`)

These changes confirm that the added tests successfully targeted previously undetected behavioral differences.

4.5.5 Analysis

S1. Effect of Test Removal Removing the five selected tests did not produce a decrease in mutation score, which remained at 84%, despite an increase in surviving mutants. This indicates that the removed tests were not uniquely responsible for killing mutants. Instead, other tests in the suite provided overlapping coverage, demonstrating redundancy.

S2. Monotonicity of Mutation Score The mutation score in this experiment is not strictly monotonic. Adding tests did not increase the score, even though additional mutants were killed. This occurs because changes in total mutant counts and coverage distribution can offset improvements, masking localized gains.

S3. Test Suite Redundancy and Uniqueness The results reveal a high degree of redundancy in the test suite, with multiple tests covering similar logic and killing the same mutants. However, the added tests demonstrate targeted effectiveness by killing specific surviving mutants, particularly in boundary conditions, buffer handling, and leniency logic. This indicates that while the suite is broadly strong, certain edge cases were previously under-tested.

4.5.6 Conclusion

The mutation score alone does not fully capture improvements in test quality. While it remained constant at 84%, the underlying mutant transitions show that targeted test additions improved fault detection. This experiment highlights both the robustness provided by redundant tests and the importance of targeted tests for addressing specific weaknesses.

5 Discussion

The full baseline largely confirmed the pre-run hypothesis. The selected Gson subsystem has very high line coverage and a strong overall mutation score, which indicates that the existing test suite is not merely executing code but is detecting many seeded faults. That is especially clear in the streaming layer, where `JsonReader` and `JsonWriter` account for most mutants and are still tested strongly.

At the same time, the calibration step mattered. The original streaming-only selection was coherent but too narrow in effective mutatable surface because several selected files were trivial under PIT's default mutators. Expanding to adjacent parser and tree-bridge classes preserved the architectural focus while improving experimental value. The final 4:09 runtime is much more appropriate for the milestone than the 55-second smoke run.

The results also show why mutation testing gives a different picture from coverage alone. Even with 96% mutated-class line coverage, 132 mutants survived in the Experiment 2 baseline. Experiment 3 then demonstrated that some of these survivors were killable with targeted tests that strengthened specific behavioral assertions. At the same time, the likely-equivalent `fillBuffer` mutant illustrates that not all surviving mutants indicate weak tests.

Together, Experiments 2 and 3 provide a clearer quality signal: the suite is already strong at subsystem level, still has actionable assertion gaps in edge-case semantics, and includes a smaller set of survivors that are better interpreted as likely equivalent rather than missing-test defects.

6 Lessons Learned

Four lessons are clear from Experiments 1–3. First, scope selection must be guided by mutatable logic rather than raw source lines alone. Second, a subsystem-focused scope can stay small enough to

run repeatedly while still generating nearly a thousand mutants if the selected classes actually contain branching behavior. Third, high coverage by itself can overstate adequacy; surviving mutants expose behavior not yet strongly asserted. Fourth, surviving-mutant triage is essential: targeted tests can eliminate real weaknesses, while explicit equivalent-mutant analysis avoids misclassifying unreachable semantic differences as test defects.

7 Conclusion

For this selected Gson subsystem, PIT setup was reproducible and produced a useful mutation-testing workflow across three experiments. The Experiment 2 baseline yielded 945 mutants with an 85.6% mutation score by the milestone formula, 86% PIT test strength, 96% line coverage, and a 4:09 wall-clock runtime. Experiment 3 then added two targeted tests that killed selected surviving mutants and improved the mutation score to 86.1% (804 killed, 130 survived, out of 957 generated mutants in the rerun).

These results show that mutation testing is practical and informative for a production Java project when the scope is chosen as a coherent feature slice with meaningful logic and a runtime in the low-minute range. The next report revisions will build on this foundation with Experiment 4 and Experiment 5 analyses using the same repository and target scope.

A PIT Configuration Excerpt

```
<profile>
  <id>milestone5-pit</id>
  <properties>
    <pitest.version>1.15.3</pitest.version>
    <pitest.target.tests>com.google.gson.*</pitest.target.tests>
    <pitest.target.class.1>com.google.gson.stream.*</pitest.target.class
        .1>
    <pitest.target.class.2>com.google.gson.JsonStreamParser</pitest.target
        .class.2>
    <pitest.target.class.3>com.google.gson.JsonParser</pitest.target.class
        .3>
    <pitest.target.class.4>com.google.gson.internal.Streams</pitest.target
        .class.4>
    <pitest.target.class.5>com.google.gson.internal.bind.JsonTreeReader</
        pitest.target.class.5>
    <pitest.target.class.6>com.google.gson.internal.bind.JsonTreeWriter</
        pitest.target.class.6>
    <pitest.target.class.7>com.google.gson.internal.bind.
        JsonElementTypeAdapter</pitest.target.class.7>
    <pitest.threads>7</pitest.threads>
    <pitest.timeout.constant>5000</pitest.timeout.constant>
    <pitest.withHistory>false</pitest.withHistory>
    <pitest.jvm.maxHeap>1024m</pitest.jvm.maxHeap>
  </properties>
  <build>
    <plugins>
      <plugin>
```

```
        <groupId>org.pitest</groupId>
        <artifactId>pitest-maven</artifactId>
        <version>${pitest.version}</version>
        <configuration>
          <targetClasses>
            <param>${pitest.target.class.1}</param>
            <param>${pitest.target.class.2}</param>
            <param>${pitest.target.class.3}</param>
            <param>${pitest.target.class.4}</param>
            <param>${pitest.target.class.5}</param>
            <param>${pitest.target.class.6}</param>
            <param>${pitest.target.class.7}</param>
          </targetClasses>
          <targetTests>
            <param>${pitest.target.tests}</param>
          </targetTests>
          <excludedTestClasses>
            <param>com.google.gson.integration.*</param>
          </excludedTestClasses>
          <mutators>
            <mutator>DEFAULTS</mutator>
          </mutators>
          <outputFormats>
            <param>HTML</param>
            <param>XML</param>
          </outputFormats>
          <jvmArgs>
            <value>-Xmx${pitest.jvm.maxHeap}</value>
          </jvmArgs>
          <withHistory>${pitest.withHistory}</withHistory>
          <timestampedReports>false</timestampedReports>
          <failWhenNoMutations>true</failWhenNoMutations>
        </configuration>
      </plugin>
    </plugins>
  </build>
</profile>
```

B Artifact Locations Used in This Draft

- Green-suite verification log: `logs/verify-gson-docker.log`
- PIT smoke log: `logs/pit-smoke-docker.log`
- PIT smoke reports: `artifacts/pit/smoke/index.html` and `artifacts/pit/smoke/mutations.xml`
- PIT baseline log: `logs/pit-baseline-docker.log`
- PIT baseline reports: `artifacts/pit/baseline/index.html` and `artifacts/pit/baseline/mutations.xml`

- PIT Experiment 3 reports: `artifacts/pit/experiment3/index.html` and `artifacts/pit/experiment3/mutations.xml`
- Experiment 3 analysis notes: `docs/Experiment3-report-1.md`
- Experiment 5 remove 5 report: `artifacts/pit/experiment5-5removed/index.html` and `artifacts/pit/experiment5-5removed/mutations.xml`
- Experiment 5 add 3 report: `artifacts/pit/experiment5-3added/index.html` and `artifacts/pit/experiment5-3added/mutations.xml`
- PIT Maven profile: `gson/gson/pom.xml`